

15. FINAL PROJECT GOAL

Build a professional AI automation platform that demonstrates:

- system design
- workflow orchestration
- AI integration
- browser automation
- backend engineering
- frontend architecture
- scalable software design

This should feel like:

- startup software
- enterprise SaaS
- modern AI automation platform

NOT a normal student CRUD project.

16. DETAILED SYSTEM DESIGN

Frontend Architecture

Core Frontend Modules

Authentication Module

Responsibilities:

- Login
- Signup
- Session management
- Route protection

Pages:

- Login Page
 - Signup Page
 - Forgot Password Page
-

Dashboard Module

Responsibilities:

- Workflow overview
- Execution analytics
- Recent activity
- Logs preview

Components:

- Sidebar
 - Top Navigation
 - Analytics Cards
 - Workflow Table
 - Activity Feed
-

Workflow Builder Module

Responsibilities:

- Drag-and-drop workflow creation
- Node linking
- Node editing
- Workflow visualization

Core Components:

- Workflow Canvas
 - Node Library
 - Node Editor
 - Connection Manager
 - Workflow Toolbar
-

Workflow Execution Module

Responsibilities:

- Execute workflows
 - Display progress
 - Stream logs
 - Stop execution
-

Integration Module

Responsibilities:

- Connect Gmail
 - Connect APIs
 - Configure integrations
 - Store credentials securely
-

Backend Architecture

API Layer

Handles:

- frontend requests
 - authentication
 - workflow APIs
 - integrations
 - execution control
-

Service Layer

Contains:

- workflow logic
 - browser automation
 - AI orchestration
 - integrations
 - execution engine
-

Agent Orchestration Layer

Coordinates:

- Planner Agent
 - Execution Agent
 - Browser Agent
 - Verification Agent
 - Summarization Agent
-

Data Layer

Handles:

- PostgreSQL
 - ORM models
 - queries
 - transactions
 - execution storage
-

17. MULTI-AGENT EXECUTION FLOW

Example Flow

User Prompt: "Search top AI internships and summarize them."

Execution Flow:

1. Planner Agent

- Understands task
- Breaks into steps

2. Browser Agent

- Opens browser
- Searches websites
- Collects data

3. Execution Agent

- Manages workflow execution
- Coordinates tasks

4. Verification Agent

- Validates extracted data
- Removes invalid results

5. Summarization Agent

- Summarizes final information
 - Returns clean output
-

18. NODE TYPES FOR WORKFLOW SYSTEM

Core Nodes

Trigger Node

Starts workflow.

Examples:

- manual trigger
 - scheduled trigger
 - webhook trigger
-

AI Node

Performs AI reasoning.

Examples:

- summarize text
 - generate response
 - analyze content
-

Browser Node

Controls browser automation.

Examples:

- open URL
 - click element
 - extract data
 - fill form
-

Integration Node

Connects external systems.

Examples:

- Gmail

- YouTube
 - APIs
 - PDFs
-

Logic Node

Controls workflow behavior.

Examples:

- conditions
 - filters
 - branching
-

Output Node

Returns final result.

Examples:

- dashboard output
 - email output
 - saved report
-

19. API DESIGN STRUCTURE

Authentication APIs

POST /auth/signup
POST /auth/login
POST /auth/logout
GET /auth/profile

Workflow APIs

POST /workflows/create
GET /workflows/list
GET /workflows/{id}
PUT /workflows/update/{id}

DELETE /workflows/delete/{id}
POST /workflows/execute/{id}

Browser APIs

POST /browser/open
POST /browser/click
POST /browser/fill
POST /browser/extract
POST /browser/screenshot

Agent APIs

POST /agents/plan
POST /agents/execute
POST /agents/verify
POST /agents/summarize

Logs APIs

GET /logs/workflow/{id}
GET /logs/execution/{id}

20. SECURITY DESIGN

Authentication Security

- JWT access tokens
 - Password hashing using bcrypt
 - Protected APIs
 - Session expiration
-

API Security

- Rate limiting
- Input sanitization
- Validation middleware

- Error handling
-

Credential Security

- Environment variables
 - Encrypted secrets
 - Secure API storage
-

Browser Security

- Sandboxed automation
 - Restricted automation permissions
 - Controlled execution environment
-

21. EXECUTION ENGINE DESIGN

Responsibilities

The execution engine:

- reads workflow JSON
 - determines execution order
 - executes nodes
 - handles failures
 - streams logs
 - stores execution state
-

Execution Lifecycle

1. Workflow Loaded
 2. Validation
 3. Node Dependency Check
 4. Agent Assignment
 5. Task Execution
 6. Logging
 7. Final Output Generation
 8. Execution Storage
-

22. DATABASE RELATIONSHIPS

users
↓
workflows
↓
workflow_executions
↓
logs

users
↓
integrations

23. WORKFLOW STORAGE FORMAT

Store workflows as JSON.

Example Structure:

```
{  
  "nodes": [],  
  "edges": [],  
  "metadata": {}  
}
```

Benefits:

- reusable
 - editable
 - scalable
 - exportable
-

24. UI DESIGN PRINCIPLES

Design Style

Use:

- modern SaaS UI

- clean spacing
 - minimal colors
 - light glassmorphism
 - dark mode first
-

UI Goals

- professional
 - not cluttered
 - workflow-focused
 - visually clean
 - scalable dashboard layout
-

25. RECOMMENDED DEVELOPMENT ORDER

DO NOT RANDOMLY SWITCH MODULES

Follow this exact order.

STEP 1 — Project Initialization

Tasks:

- Setup GitHub repository
- Create frontend project
- Create backend project
- Setup PostgreSQL
- Configure environment variables
- Setup Docker basics

Goal: Stable full-stack base.

STEP 2 — Authentication System

Tasks:

- Signup API
- Login API
- JWT implementation
- Password hashing

- Protected routes
- Session handling

Goal: Secure authentication layer.

STEP 3 — Dashboard UI

Tasks:

- Sidebar
- Navbar
- Dashboard cards
- Workflow listing
- Analytics placeholders

Goal: Professional SaaS layout.

STEP 4 — Workflow Database Models

Tasks:

- Workflow schema
- Execution schema
- Logs schema
- Relationships

Goal: Stable data structure.

STEP 5 — Workflow APIs

Tasks:

- Create workflow
- Update workflow
- Delete workflow
- Execute workflow
- Save workflow

Goal: Core workflow backend ready.